

SmartFix API Contract v2

SmartFix — API Contract

Audience: Frontend (React) developer **Backend:** FastAPI (Python) **Status:** Draft v2 — see “Stability notes” at the end before assuming any field is locked. **Last updated:** 2026-05-02 (post Tecdia progress meeting)

1. Overview

The SmartFix backend exposes a small REST API with four concerns:

1. **Auth** — two flows: workers select a domain at a dedicated workstation; managers/admins use magic-link login on their own devices.
2. **RAG query** — workers ask questions about a specific machine; the backend retrieves relevant manual excerpts and runs them through an LLM. Each answer is rated for severity.
3. **Admin** — admins upload new machine PDFs; the backend parses, chunks, embeds, and indexes them in the background.
4. **Alerts** — high-severity answers ($\text{severity} \times \text{machine significance} \geq \text{threshold}$) auto-create alert records and notify managers by email.

Roles: `worker` (read-only, scoped to a domain) and `admin` (read + manage machines + view alerts). Worker domain is selected at login. Admin role is determined server-side based on the email used in magic-link login.

2. Conventions

- **Base URL:** `/` (FastAPI app root). When deployed, the frontend will configure this via an env var, e.g. `API_BASE=https://api.smartfix.tecdia.com`.
- **Content type:** `application/json` for all requests/responses unless noted (the only exception is `multipart/form-data` for PDF uploads).
- **Authentication:** session cookie set during `/auth/worker-session` (workers) or `/auth/verify` (admins). Cookies are `httpOnly`, `Secure` (in prod), `SameSite=Lax`. The frontend `fetch` calls must include `credentials: 'include'`.

- **Error envelope:** every non-2xx response uses this shape:

```
{ "detail": "human-readable message", "code": "machine_readable_code" }
```

- **Timestamps:** ISO 8601 UTC, e.g. `2026-05-02T16:47:11Z` .
- **Unknown fields:** the frontend should ignore unknown response fields (so additive backend changes don't break the client).

3. Authentication flows

Two separate flows feed into the same `/auth/me` shape.

3.1 Worker flow (dedicated workstations)

1. Worker sits at a workstation, opens the app
2. Sees the "Select Your Expertise Domain" screen
3. `POST /auth/worker-session { domain: "Manufacturing" }`
 - └ Server: validates domain, sets ``worker_session`` cookie (12h TTL)
 - └ Returns `{ authenticated, role: "worker", domain }`
4. React app calls `GET /auth/me` to confirm and renders worker UI
5. Session lasts 12 hours (one shift). After expiry → back to domain selector.
6. `POST /auth/logout` to clear the session.

No email, no password, no token. The workstation is trusted by physical presence on the factory floor.

3.2 Admin / manager flow (magic-link, own device)

1. Manager opens app on their own device, sees email input
2. `POST /auth/request-link { email }`
 - └ Server: if email allowlisted → email a token link
3. Manager opens email, taps link
4. `GET /auth/verify?token=...`
 - └ Server: validates token, creates session, sets cookie
 - └ Server: 302 redirects to /
5. React app loads, calls `GET /auth/me`
 - └ Renders admin panel (`role === "admin"`)
6. Session lasts 30 days
 - └ Any 401 response → React routes back to login screen

Notes: - The role is decided at step 4 by checking whether the email is in the server's `ADMIN_EMAILS` allowlist. - Tokens expire 15 minutes after issuance, are single-use, and are invalidated after

consumption. - This flow is **not yet wired to a real email provider** — currently a stub. Awaiting Tecdia domain decision for Resend/SMTP.

4. Endpoint reference

4.1 Auth

POST /auth/worker-session

Open a worker session by selecting a domain. No email needed.

- **Auth required:** No

Request

```
{ "domain": "Manufacturing" }
```

`domain` must be one of: `"General"`, `"Manufacturing"`, `"Additive Manufacturing"`, `"Fabrication"`, `"Automation"`, `"Heavy Machinery"`, `"All Access"`.

Response — 200

```
{
  "authenticated": true,
  "role": "worker",
  "domain": "Manufacturing",
  "email": null
}
```

Sets cookie `worker_session=...; HttpOnly; SameSite=Lax; Max-Age=43200` (12h).

Response — 400

```
{ "detail": "Invalid domain 'Foobar'", "code": "invalid_domain" }
```

POST /auth/request-link

Send a magic-link email to a manager.

- **Auth required:** No

Request

```
{ "email": "kana@tecdia.com" }
```

Response — 200 (always, regardless of allowlist match)

```
{ "ok": true }
```

Response — 422 (malformed email)

```
{ "detail": "Invalid email format", "code": "invalid_email" }
```

The server intentionally returns 200 even for non-allowlisted emails, so attackers can't enumerate valid users. The actual email is only sent to allowlisted addresses.

GET /auth/verify?token={token}

Consume a magic-link token, create a session, set cookie, redirect to app.

- **Auth required:** No (this endpoint *creates* the session)

Query params

Name	Type	Required	Description
token	string	Yes	Token received in the email link.

Response — 302 redirect

- On success: `Set-Cookie: session=...; HttpOnly; Secure; SameSite=Lax; Max-Age=2592000` and redirect to `/`.
- On invalid/expired/used token: redirect to `/?login_error=expired`. The frontend reads the query param and shows an error.

GET /auth/me

Return the currently logged-in user. **The frontend should treat this as the single source of truth** — never assume how the user authenticated.

- **Auth required:** Yes

Response — 200 (worker)

```
{
  "authenticated": true,
  "role": "worker",
  "domain": "Manufacturing",
  "email": null,
  "session_expires_at": "2026-05-03T04:47:09Z"
}
```

Response — 200 (admin)

```
{
  "authenticated": true,
  "role": "admin",
  "domain": "All Access",
  "email": "kana@tecdia.com",
  "session_expires_at": "2026-06-01T16:46:59Z"
}
```

`role` is one of: `"worker" | "admin"`. `domain` is always populated. Admins always get `"All Access"`.

Response — 401

```
{ "detail": "Not authenticated", "code": "unauthenticated" }
```

Frontend calls this on app load. On 401, route to login screen (which shows either the domain selector or the email input depending on the workstation/device context).

POST /auth/logout

Invalidate the current session.

- **Auth required:** Yes
- **Request body:** *(empty)*

Response — 200

```
{ "ok": true }
```

The server clears `worker_session` and any admin session cookies.

4.2 Worker + Admin (any authenticated user)

GET /machines

List machines available to query against. Workers receive only machines whose `category` matches their domain (or all if domain is `"All Access"`); admins receive all machines.

- **Auth required:** Yes (any role)

Response — 200

```
{
  "machines": [
    {
      "id": "INJECTION_MOLDING_MACHINE",
      "display_name": "Injection Molding Machine",
      "chunk_count": 10,
      "description": "Tecdia injection molding line – IMM-750 series.",
      "category": "Manufacturing",
      "significance": 5,
      "icon": null
    },
    {
      "id": "LASER_CUTTING_MACHINE",
      "display_name": "Laser Cutting Machine",
      "chunk_count": 17,
      "description": "Tecdia precision laser cutter – LC-2040 series.",
      "category": "Fabrication",
      "significance": 4,
      "icon": null
    }
  ]
}
```

Field reference

Field	Type	Notes
id	string	Machine slug. Use this for <code>machine_filter</code> in <code>/query</code> .
display_name	string	Human-readable name for UI.
chunk_count	int	Number of indexed chunks. Useful for admin metrics; can be hidden in worker UI.
description	string	Free text shown on machine card. May be empty.
category	string	One of the <code>domain</code> values. Used for worker access control.
significance	int	1–5. Business impact of the machine. Used in the alert score (<code>severity × significance</code>).
icon	string null	Filename of uploaded icon, or null. Frontend renders default icon when null.

Use `id` as the value sent in `machine_filter` to `/query` . Use `display_name` in the UI.

POST `/query`

Ask a question about a specific machine’s documentation.

- **Auth required:** Yes (any role)

Request

```
{
  "question": "What does error E-04 mean?",
  "machine_filter": "INJECTION_MOLDING_MACHINE",
  "history": [
    { "role": "user", "content": "What does error E-02 mean?" },
    { "role": "assistant", "content": "Error E-02 indicates ..." }
  ]
}
```

Field	Type	Required	Notes
question	string	Yes	Max length 500 chars.
machine_filter	string	Yes	Must match an <code>id</code> from <code>/machines</code> .
history	array	No	Last N turns. Server caps at 8 messages. Each item: <code>{role: "user" "assistant", content}</code> .

Response — 200

```
{
  "status": "success",
  "answer": "Error E-04 indicates the clamping force has not reached the target
            setpoint (page 5)...",
  "sources": [
    { "document": "INJECTION_MOLDING_MACHINE.pdf", "page": 5 },
    { "document": "INJECTION_MOLDING_MACHINE.pdf", "page": 6 }
  ],
  "severity_level": 4,
  "alert_score": 20,
  "machine_significance": 5,
  "alert_fired": true
}
```

Field reference

Field	Type	Notes
status	enum	"success" \ "not_found" \ "error"
answer	string	Human-readable answer with inline page citations.
sources	array	List of {document, page} for the chunks the answer was grounded on.
severity_level	int	1–5. LLM-assessed urgency of the user’s problem (see severity rubric below).
alert_score	int	$\text{severity_level} \times \text{machine_significance}$. Range 1–25.
machine_significance	int	Echoed from the machine. Useful for the UI to render alert badges in context.
alert_fired	bool	True if the backend auto-created an alert record (score $\geq$ threshold, default 12).

status values:

Value	Meaning
success	answer and sources are populated.
not_found	No relevant chunks for this question/machine. sources is empty; answer is generic.
error	LLM service unavailable. sources is empty.

Severity rubric (assessed by the LLM per query):

Level	Meaning
1	Informational, no action needed
2	Minor adjustment, machine still operational
3	Degraded performance, plan a fix soon
4	Production impact, fix urgently
5	Production halted or safety risk, immediate intervention

Response — 403 (worker queried a machine outside their domain)

```
{
  "detail": "Machine 'LASER_CUTTING_MACHINE' is not in your domain
            'Manufacturing'",
  "code": "access_denied"
}
```

Response — 422 (validation, e.g. question too long)

```
{ "detail": "question must be ≤500 chars", "code": "validation_error" }
```

4.3 Admin only

All admin endpoints require `role: "admin"` on the session. A worker hitting these gets `403`.

POST /admin/machines

Upload a new PDF for ingestion. Returns immediately with a `job_id`; ingestion runs in the background.

- **Auth required:** Yes (admin)
- **Content type:** `multipart/form-data`

Form fields

Field	Type	Required	Notes
file	file	Yes	PDF only. Max 50 MB.
machine_id	string	Yes	Slug, <code>[A-Z0-9_]+</code> . Must not already exist.
display_name	string	Yes	Human-readable, e.g. <code>"Injection Molding Machine"</code> .
description	string	No	Free text shown on machine card. Defaults to empty.
category	string	No	One of the domain values. Defaults to <code>"General"</code> .
significance	int	No	1–5. Business impact of the machine. Defaults to 3.
icon	file	No	Optional icon image (PNG/SVG/JPG).

Response — 202 Accepted

```
{ "job_id": "job_abc123", "status": "queued" }
```

Response — 409

```
{ "detail": "Machine already exists", "code": "machine_exists" }
```

Response — 413

```
{ "detail": "File exceeds 50 MB", "code": "file_too_large" }
```

Response — 422

```
{ "detail": "significance must be 1-5", "code": "validation_error" }
```

GET /admin/jobs/{job_id}

Poll an ingestion job's status.

- **Auth required:** Yes (admin)

Response — 200

```
{
  "job_id": "job_abc123",
  "machine_id": "NEW_MACHINE",
  "status": "embedding",
  "step": "Embedding 42 of 87 chunks",
  "progress": 0.48,
  "started_at": "2026-04-29T10:01:23Z",
  "finished_at": null,
  "error": null
}
```

Job status lifecycle

```
queued → parsing → chunking → embedding → indexing → done
                                         ↓
                                         failed
```

When `status === "failed"`, `error` contains the message to show the admin. When `status === "done"`, `finished_at` is set and the new machine appears in `/machines`.

Frontend should poll this every ~2 seconds while `status` is not `done` or `failed`.

GET /admin/machines

List machines with extra admin metadata.

- **Auth required:** Yes (admin)

Response — 200

```
{
  "machines": [
    {
      "id": "INJECTION_MOLDING_MACHINE",
      "display_name": "Injection Molding Machine",
      "chunk_count": 10,
      "description": "Tecdia injection molding line – IMM-750 series.",
      "category": "Manufacturing",
      "significance": 5,
      "icon": null,
      "uploaded_at": "2026-04-15T08:21:00Z",
      "uploaded_by": "alice@tecdia.com.ph",
      "pdf_size_bytes": 2451200
    }
  ]
}
```

Same fields as `/machines` plus three admin-only fields (`uploaded_at` , `uploaded_by` , `pdf_size_bytes`).

DELETE `/admin/machines/{machine_id}`

Remove a machine and all its chunks from the index. Also clears its metadata.

- **Auth required:** Yes (admin)

Response — 200

```
{ "ok": true, "deleted_chunks": 16 }
```

Response — 404

```
{ "detail": "Machine not found", "code": "not_found" }
```

4.4 Alerts (admin only)

Alerts are auto-created by `/query` whenever `alert_score ≥ ALERT_THRESHOLD` (default 12). When fired, the backend (in production) emails the addresses in `ADMIN_EMAILS` .

GET `/admin/alerts`

List all alerts, newest first.

- **Auth required:** Yes (admin)

Response — 200

```
{
  "alerts": [
    {
      "alert_id": "alert_c956af33",
      "machine_id": "INJECTION_MOLDING_MACHINE",
      "score": 20,
      "severity_level": 4,
      "machine_significance": 5,
      "question": "What does error code E-04 indicate and how do I resolve it?",
      "answer_excerpt": "Error code E-04 indicates that the measured tie bar stretch...",
      "email_notified": true,
      "notified_at": "2026-05-02T16:47:11Z"
    }
  ],
  "threshold": 12
}
```

Field reference

Field	Type	Notes
alert_id	string	Stable ID for this alert.
machine_id	string	Which machine the alert was for.
score	int	<code>severity_level × machine_significance</code> .
severity_level	int	1–5.
machine_significance	int	1–5.
question	string	The full user question that triggered the alert.
answer_excerpt	string	First ~280 chars of the LLM answer.
email_notified	bool	Whether the alert email was sent. False if email pipeline misfired.
notified_at	string	ISO timestamp of alert creation.

The top-level `threshold` echoes the current alert threshold so the UI can show “Alerts ≥ 12 of 25”.

DELETE `/admin/alerts`

Clear all alert history. Maps to the “Clear All History” button.

- **Auth required:** Yes (admin)

Response — 200

```
{ "ok": true, "cleared": 7 }
```

POST /admin/alerts/test

Inject a synthetic alert. Useful for verifying the email pipeline once it's wired.

- **Auth required:** Yes (admin)

Response — 201

```
{
  "alert_id": "alert_xyz",
  "machine_id": "INJECTION_MOLDING_MACHINE",
  "score": 15,
  "severity_level": 5,
  "machine_significance": 3,
  "question": "TEST – synthetic alert for setup verification",
  "answer_excerpt": "This is a test alert. Email pipeline can be verified here.",
  "email_notified": false,
  "notified_at": "2026-05-02T16:50:00Z"
}
```

4.5 Health

GET /health

Liveness probe.

- **Auth required:** No

Response — 200

```
{ "status": "ok", "version": "0.1.0" }
```

5. Standard error codes

HTTP	code	When
400	bad_request	Malformed body
400	invalid_domain	Worker session opened with an unknown domain value
401	unauthenticated	No session, expired session, or invalid cookie
403	forbidden	Authenticated but role insufficient (worker hitting <code>/admin/*</code>)
403	access_denied	Worker queried a machine outside their domain
404	not_found	Resource doesn't exist
409	machine_exists	Duplicate <code>machine_id</code> on upload
413	file_too_large	PDF over 50 MB
422	validation_error	Pydantic validation failed (per-field details in <code>detail</code>)
429	rate_limited	Reserved for future rate limiting
500	internal_error	Unhandled server error (with friendly generic message)

6. Recommended build order for the frontend

Each step is independently shippable.

1. **Login screen** + `/auth/me` — domain selector for workers, email input for admins. Build everything else against the `/auth/me` shape; never assume how the user authenticated.
2. **Chat UI** against `/query` + `/machines`. Render machine cards with category/significance, send queries with `history`. Surface `severity_level` and `alert_score` in the UI if useful.
3. **Admin panel** behind a `role === "admin"` gate. Upload form (with description / category / significance slider / icon) + progress polling + machine list/delete.
4. **Alerts dashboard** in the admin panel using `/admin/alerts`.

7. In-chat memory (15-minute idle expiry)

The frontend manages chat history client-side, so the backend stays stateless. Implementation guidance:

- Store `{history: Turn[], lastActivity: number}` in `localStorage` under key `smartfix.history`.
- On app load, if `Date.now() - lastActivity > 15 * 60 * 1000`, clear the stored history.

- On every `/query` request, include the stored history in the request body.
- Append both the user turn and the assistant turn after each response, then update `lastActivity`.
- Provide a “Start over” button that clears the storage and the in-memory state.

This survives accidental page reload, persists across phone sleep cycles within the 15-minute window, and auto-clears after a shift change.

8. Stability notes

What’s locked in (won’t change)

- `/auth/me` response shape — this is the contract between the auth flow and the rest of the app. Auth method may evolve; the response shape will not.
- Role enum (`worker` | `admin`); future roles will be additive.
- Domain enum (the seven values in §4.1); future domains will be additive.
- Request fields on `/query` (`question` , `machine_filter` , `history`).
- Response status enum on `/query` (`success` | `not_found` | `error`).
- `severity_level` is always 1–5 integer. Range will not change.
- `machine_significance` is always 1–5 integer. Range will not change.
- Async ingestion pattern (`POST` returns `job_id` ; poll `/admin/jobs/{id}`).
- Job status lifecycle.
- Error envelope shape (`{detail, code}`).

What may extend (additive — old code still works)

- `sources[].text` field on `/query` responses (for source previews).
- Display name / last login on `/auth/me`.
- Finer-grained progress on `/admin/jobs/{id}`.
- New domain values added to the worker domain selector.
- New fields on alert records (e.g., `acknowledged_by` , `acknowledged_at`).

What could shift if cloud topology changes

- **Cookies vs Bearer tokens:** if the deployed frontend ends up on a different domain than the API, the auth credential may switch from a cookie to a `Authorization: Bearer ...` header. To future-proof, hide auth credential handling behind a `getAuthHeaders()` / `credentials: 'include'` wrapper in the fetch layer.

What's open / pending Tecdia decisions

- **Severity rubric wording** — defaults provided in §4.2, awaiting validation from Tecdia ops team.
- **Significance rubric** — admins set 1–5 per machine. Need Tecdia examples per level.
- **Alert threshold** — defaults to 12. Tunable via `ALERT_THRESHOLD` env var.
- **Email provider** for magic links and alert notifications (Resend vs Tecdia SMTP). No contract impact.
- **Vector DB choice** (Qdrant Cloud vs ChromaDB on Tecdia local server). Internal to backend; no contract impact.
- **Worker session length** — currently 12h. Tecdia may want longer/shorter.

What's deliberately deferred (not in v2)

- Real magic-link email send (currently `/auth/request-link` is a stub — returns 200 without sending).
- Real alert email send (currently `email_notified` is a stub flag).
- Persistence of worker sessions, machine metadata, and alerts (currently in-memory; lost on restart).

End of contract — v2